SKILLNAVORA · AI TOOLS · LOVABLE TUTORIAL

Lovable: Complete Tutorial

Build web apps by chatting with Lovable: prompts, design, Supabase data and login, GitHub sync, publishing, custom domains, templates and a project.

Edition 2026 · skillnavora.in

How to use this guide

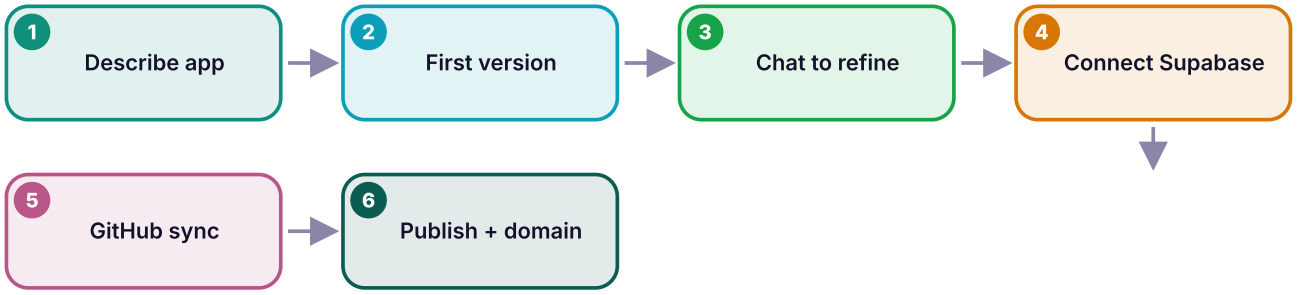
Each class has goals, concepts with Indian examples, key takeaways and a practice exercise, from basic to advanced. Finish with the final project to build something you can show in interviews.

	Class	Level
Class 1	**Lovable basics**	Basic
Class 2	**Write prompts that build well**	Basic
Class 3	**Data and login with Supabase**	Intermediate
Class 4	**GitHub sync and publishing**	Intermediate
Class 5	**Prompt and component templates**	Intermediate
Class 6	**Design to build: screens, components and visual edits**	Intermediate
Class 7	**Authentication, roles and Row Level Security that actually protects data**	Intermediate
Class 8	**Payments, email, AI features and third-party APIs**	Intermediate
Class 9	**Testing, quality and the security scan**	Advanced
Class 10	**GitHub, collaboration and moving to production**	Advanced
Class 11	**Playbook: from idea to launched MVP in two weeks**	Advanced
Class 12	**Troubleshooting and FAQ: Lovable**	Advanced
Class 13	**Practice challenges with solutions: Lovable**	Advanced
Class 14	**Project, quality and limits**	Advanced
	Toolkit: templates, lovable cheat sheet, glossary, 12 interview questions, further reading	

BASIC · CLASS 1

Lovable basics

Understand what Lovable builds and how credits work.



Lovable flow

- Sign in at lovable.dev → **New project** → write your prompt. The app appears in a live preview next to the chat.
- Edit by chat, use **Select/Edit** on an element to change just that part, and **Version history** to restore.
- Plans use message credits: batch your requests and be specific (check the pricing page for current limits).

Lovable at a glance

Part	Role
Chat / prompt panel	Describe changes; ask questions in chat mode and build in agent mode (names may vary)
Live preview	See and click the app; use visual edit tools for small text and style changes
Code view and GitHub	Inspect and sync the code to a repository
Supabase integration	Database, authentication, storage and server functions
Project knowledge	Instructions and context that the AI keeps in mind (brand rules, constraints)
Publish and domains	Deploy to a public URL and connect a custom domain
Security scan	Automated checks for common issues (confirm current features)

Check before you build: app names, plan limits, prices and menu labels change often. Use this lesson for the method and confirm the current screens and limits in the official docs.

Key takeaways

- Chat builds, preview shows.
- Edit single elements.
- Credits: be specific.

Practice exercise: Create a project and change one element using Select/Edit.

BASIC · CLASS 2

Write prompts that build well

Use a mini-PRD for the first prompt.

```
App: Class Booking for a yoga studio in Pune.
Users: members and studio admin.
Pages: Home, Classes (list + filter), Class detail with Book button, My bookings, Admin
(add/edit classes, see bookings).
Style: calm, green and cream, rounded cards, mobile-first.
Data: classes (title, trainer, date/time, seats), bookings (member, class).
Rules: a class can't be booked when full; confirmation message after booking.
```

- Name pages, data, rules and style; ask for sample data.
- Then iterate one change at a time: "Add a pricing page with 3 plans."
- Paste screenshots or reference sites for visual direction.

Prompt formula: ROLE, GOAL, SCREENS, DATA, RULES, STYLE

```
You are building a clinic appointment app.
Goal: patients book a 15-minute slot with a doctor and get a confirmation.
Screens: Home, Doctor list, Doctor detail with calendar, Booking form, My bookings, Admin
(doctors, slots).
Data: doctors(name, speciality, fee), slots(doctor, start, status), bookings(patient, slot,
status).
Rules: no double booking; patients see only their bookings; admins manage slots.
Style: calm green, large text, mobile first, English and Hindi labels.
```

- Start with **layout and mock data**, then connect real data.
- One feature per prompt; test; then move on.
- Use **screenshots or sketches** for layout and say 'keep everything else unchanged'.

Key takeaways

- Mini-PRD first prompt.
- One change at a time.
- Screenshots help.

Practice exercise: Write a mini-PRD prompt and build the first version.

INTERMEDIATE · CLASS 3

Data and login with Supabase

Add a database and user accounts.

1. Connect **Supabase** (button in the project); create or link a project.
2. Ask: "Create tables for classes and bookings with row-level security; members see only their own bookings; admins see all."
3. Ask for sign-up/login (email and Google) and an admin role.
4. Test as a member and as an admin; check that one user can't see another's data.
5. Keep secret keys in the backend/edge-function secrets, never in the front end.

Supabase concepts in plain words

Concept	Meaning
Tables	Where records live (like spreadsheets)
Authentication	Sign-up, login, password reset, social login
Row Level Security (RLS)	Rules that decide which rows each user may read or change
Policies	The actual RLS rules, for example 'a user can see rows where user_id equals their ID'
Storage	Buckets for files and images
Edge Functions	Small server-side code for secrets, payments and third-party calls
Secrets	Keys stored on the server side, never in the front-end

> **Never** put secret keys in front-end code. Use edge functions for payment secrets and private API keys; the public (anon) key is designed to be exposed but is only safe when RLS is correct.

Key takeaways

- Supabase = database + auth.
- Row-level security protects data.
- Test with two users.

Practice exercise: Add login and two tables; prove users can't see each other's bookings.

INTERMEDIATE · CLASS 4

GitHub sync and publishing

Own your code and go live.

1. **GitHub:** connect, create a repo; changes sync both ways, so a developer (or Claude Code/Codex) can continue the project.
2. **Publish:** click Publish for a public URL.
3. **Custom domain:** project settings → Domains → add your domain; copy the DNS records to your registrar; wait for HTTPS.
4. **After launch:** add analytics (GA4/Clarity), SEO titles and a privacy page.

Publishing checklist

- Test sign-up, login, logout, password reset and the main flow with two accounts.
- Check mobile layout and speed.
- Set page titles, descriptions and share images.
- Connect a custom domain with HTTPS.
- Add privacy policy and terms pages; add cookie notice if needed.

Key takeaways

- Sync to GitHub for ownership.
- Domains need DNS records.
- Add analytics and privacy page.

Practice exercise: Publish your app on a custom domain or subdomain.

INTERMEDIATE · CLASS 5

Prompt and component templates

Copy these starters.

Need	Prompt
Landing page	"Add a hero, 3 benefits, testimonials, pricing and FAQ; make the CTA 'Book a free call'."
Contact form	"Add a form (name, phone, message) that saves to Supabase and emails me."
Dashboard	"Add an admin dashboard with bookings per week and a table with filters."
Payments	"Add a Razorpay (test mode) checkout for class packs; confirm on webhook."
Fix	"The mobile menu overlaps the logo; fix without changing other pages."

Key takeaways

- Name the exact change.
- Say what must not change.
- Ask for fixes by symptom.

Practice exercise: Apply 3 templates to your app.

INTERMEDIATE · CLASS 6

Design to build: screens, components and visual edits

Get the UI you want without wasting credits.

- **Start from structure:** list pages and sections first; ask for a layout with sample content.
- **Design system prompts:** define colours, fonts, spacing, border radius, button style, and ask for reusable components (Button, Card, Modal, Form).
- **Reference images:** upload a screenshot of a layout you like; ask to match the structure, not copy branding.
- **Visual edits:** use click-to-edit for text, colours and spacing to save prompts.
- **Responsive:** ask to check mobile, tablet and desktop; test in the preview's device modes.
- **Accessibility:** request labels, sufficient contrast, keyboard navigation and alt text.
- **Content:** provide real copy; avoid lorem ipsum.

Prompt	Result
Create a reusable PricingCard component with title, price, features and a CTA; show three plans	A consistent component used three times
Make the header sticky and collapse the menu into a hamburger on mobile; do not change other pages	Targeted edit
Use a 4-column grid on desktop, 2 on tablet, 1 on mobile for product cards	Responsive layout

Key takeaways

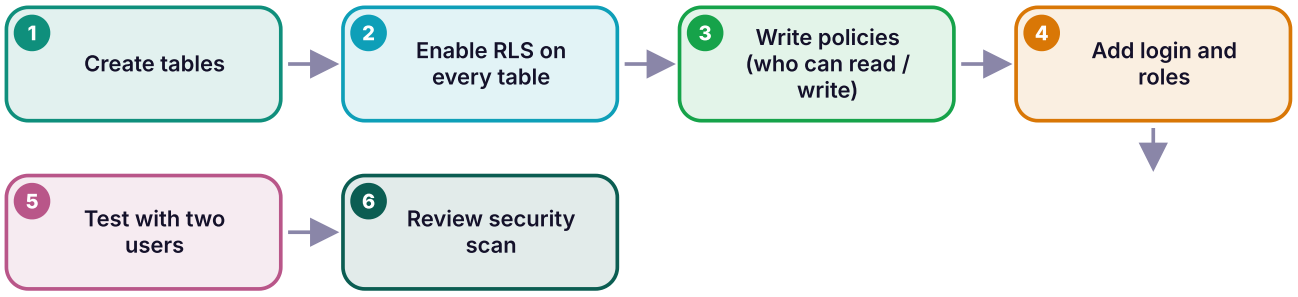
- Define the design system early.
- Use visual edits for small changes.
- Test on mobile every time.

Practice exercise: Write a design-system prompt (colours, type, components) and build a three-page site from it.

INTERMEDIATE · CLASS 7

Authentication, roles and Row Level Security that actually protects data

Set up logins and permissions correctly.



A safe data-access routine

Scenario	Policy idea
Users see only their own records	Allow select, insert, update where user_id = the logged-in user
Public catalogue	Allow select for everyone; insert and update only for admins
Team data	Allow access if the user is a member of the team (join table)
Admin panel	Store roles in a separate table; check role in the policy, not in the front-end
Private files	Storage policies by folder or owner

- **Roles:** keep roles in a protected table; never trust a role value sent from the browser.
- **Test attacks:** try to read another user's record by changing an ID in the URL or request.
- **Audit:** turn on logging and review the security scan results before launch.

Key takeaways

- Enable RLS on every table.
- Roles live on the server side.
- Test by trying to break your own app.

Practice exercise: Write RLS policies in plain English for a booking app with patients, doctors and admins.

INTERMEDIATE · CLASS 8

Payments, email, AI features and third-party APIs

Add real-world features safely.

Feature	How	Safety
Payments	Hosted checkout (Razorpay or Stripe) via an edge function; webhook marks order as paid	Never trust the browser; verify webhook signatures; store payment IDs
Transactional email	Provider API called from an edge function	Rate limits; no secrets in front-end
AI features (chat, summaries)	Call a model API from an edge function with your key	Limit usage per user; filter inputs; log costs
Maps and location	Maps API with restricted keys	Restrict by domain; consent for location
Analytics	Events to an analytics tool	Avoid personal data in events
WhatsApp / SMS	Provider API via edge function	Opt-in, templates, DND rules

For each integration, write the data flow: user action → front-end → edge function → provider → database → confirmation, and list what can fail.

Key takeaways

- Use edge functions for secrets.
- Verify webhooks and store IDs.
- Limit AI and email usage per user.

Practice exercise: Add a payment flow design (steps and failure cases) for a paid plan.

ADVANCED · CLASS 9

Testing, quality and the security scan

Ship fewer bugs.

Check	Why	How
Secrets never in code or chat	Keys in prompts or repos leak and get abused	Use the Secrets panel or environment variables; rotate any key you pasted by mistake
Authentication and roles	Anyone with the link could see private data	Require login for private pages; check roles on the server, not only in the UI
Input validation	Bad input causes errors and attacks	Validate types and lengths; escape output; use parameterised queries
Row-level or per-user data rules	Users must only see their own data	Define rules in the database; test with two accounts
Rate limits and abuse control	Bots can run up costs	Limit sign-ups, form posts and AI calls; add CAPTCHA if needed
Payment handling	Card data is regulated	Use a hosted checkout (for example Razorpay, Stripe); never store card numbers
Privacy and consent	Personal data laws apply (DPDP Act)	Collect minimum data, show a privacy notice, allow deletion
Backups and recovery	Mistakes and bad deploys happen	Use checkpoints, version control and database backups; test a restore

Test type	Examples
Happy path	Sign up, book, pay, confirm
Edge cases	Empty fields, long text, wrong dates, double clicks, back button
Permissions	User A tries to open User B's data
Mobile and slow network	Real device on 4G
Regression	Re-test earlier features after each change

Ask the AI to write a short test checklist and to explain each fix it makes. Keep a list of known issues.

Key takeaways

- Test happy path, edge cases and permissions.
- Do a security review before publishing.
- Keep known issues documented.

Practice exercise: Create a test checklist with 15 cases for your app and run it.

ADVANCED · CLASS 10

GitHub, collaboration and moving to production

Work with engineers and grow beyond the prototype.

- **GitHub sync:** connect a repository; the code stays in sync; engineers can review, branch and extend. Avoid editing the same files in two places at once.
- **Environments:** separate Supabase projects for development and production; do not test on live data.
- **Handoff package:** repository, README, environment variable names, database schema and RLS policy list, user flows, known risks and next steps.
- **When to rebuild:** if performance, compliance or complex logic needs exceed the generated code, plan a clean build with engineers using the prototype as a specification.
- **Maintenance:** update dependencies, monitor errors, rotate keys, review RLS after schema changes.

Stage	Goal	Exit criteria
Prototype	Learn	5 users complete the main flow
MVP	Real use	Auth, RLS, payments, monitoring, support process
Scale	Reliability	Engineering ownership, testing, SLAs

Key takeaways

- Sync to GitHub and keep environments separate.
- Prepare a handoff package.
- Plan a rebuild when needs outgrow the prototype.

Practice exercise: Prepare a handoff document for an app you built in Lovable.

ADVANCED · CLASS 11

Playbook: from idea to launched MVP in two weeks

Follow a time-boxed plan.

Day	Focus	Output
1	Problem, users, success metric; write the mini-PRD	PRD and prompt pack
2	Layout and design system with sample data	Clickable UI
3 to 4	Data model, auth and RLS	Working login and data
5 to 6	Core flow end to end	Main use case works
7	Payments or integrations (if needed)	Test payments
8	Quality pass: test, security scan, mobile	Bug list and fixes
9	Content, SEO basics, privacy and terms	Ready pages
10	Beta with 10 users, collect feedback	Insights
11 to 12	Fix top issues, add analytics	Improved MVP
13	Publish with custom domain	Live URL
14	Measure and decide: iterate, hand off or stop	Decision memo

Define success first (for example 30 sign-ups and 10 completed bookings), so you know whether the MVP worked.

Key takeaways

- Time-box the MVP and define success first.
- Beta with real users before polishing.
- End with a decision: iterate, hand off or stop.

Practice exercise: Create your 14-day plan and run the first three days.

ADVANCED · CLASS 12

Troubleshooting and FAQ: Lovable

Fix common problems quickly.

Problem	Likely cause	Fix
Data not showing	RLS blocks reads or table empty	Check policies; test with the logged-in user; add seed data
Login fails or loops	Redirect URLs or confirmation settings	Check auth settings and URLs; test in a private window
Payment shows paid but order not updated	Webhook not configured or not verified	Set the webhook; verify signature; check logs
UI changed unexpectedly	Broad prompt	Use targeted prompts, restore a version, say what must not change
Credits used fast	Many large rewrites	Plan first, use visual edits, work in slices
Slow on mobile	Large images or lists	Compress images, paginate, lazy load

Debug routine: describe steps, expected vs actual, include error text or screenshot, ask for the cause first, then the fix, then re-test with two accounts.

Key takeaways

- Check RLS first for missing data.
- Use targeted prompts and versions.
- Verify payment webhooks.

Practice exercise: Create a policy mistake on purpose and fix it using the debug routine.

ADVANCED · CLASS 13

Practice challenges with solutions: Lovable

Test yourself on realistic tasks.

Challenge 1

Build a booking app where users see only their bookings.

Solution: Tables for services and bookings with user_id; enable RLS; policy select/insert/update where user_id = auth user; test with two accounts.

Challenge 2

Add an admin dashboard.

Solution: Roles table; policies check role in the database; separate admin routes; test that a normal user cannot read admin data.

Challenge 3

Add a paid plan.

Solution: Edge function creates a hosted checkout session; webhook updates the subscription row after signature verification; UI reads plan from the database.

Challenge 4

Prepare for launch.

Solution: Security scan, mobile test, privacy and terms pages, custom domain, analytics events, support email, and a backup plan.

Key takeaways

- Practise RLS, roles, payments and launch.
- Test with multiple accounts.
- Prepare legal and support pages.

Practice exercise: Create your own three challenges with solutions.

ADVANCED · CLASS 14

Project, quality and limits

Ship and maintain.

Project: booking or lead app

Mini-PRD → build → Supabase + login → payments (test) → GitHub → domain → 10-case test list → 3 real users → fix list.

- **Quality:** check mobile, empty states, errors, accessibility; review security (RLS, secrets).
- **Limits:** complex back-end logic may need a developer or Claude Code/Codex after GitHub sync.

Credits and cost awareness

- Each prompt uses credits; large rewrites cost more. Be specific, work in slices, and avoid 'rebuild everything'.
- Use chat or plan mode for questions and planning without changing code (confirm current names).
- Check plan limits for projects, users, storage and custom domains.

Key takeaways

- Test with real users.
- Check security basics.
- Move to a coding agent for complex logic.

Practice exercise: Complete the project and write a 1-page launch checklist.

TOOLKIT

Templates

Lovable project brief

Field	Fill in
Users and roles	
Pages	
Data tables	
Rules	
Integrations	
Domain + analytics	
10 test cases	

TOOLKIT

Lovable cheat sheet

Term / tool	Meaning / use
Preview	Live running app
Select/Edit	Edit one UI element by chat
Supabase	Database + login backend
RLS	Row-level security: who can see which rows
GitHub sync	Two-way code sync
Publish	Deploy to a public URL
Prompt formula	Role, goal, screens, data, rules, style
Security	RLS on, secrets on server, test two users
MVP plan	14 days with beta and a decision
Handoff	Repo, README, schema, RLS list, risks

Glossary

RLS	Row Level Security, database rules per user
Policy	A rule that allows or denies actions on rows
Edge Function	Small server-side function
Anon key	Public key for the front-end, safe only with RLS
Service role key	Powerful secret key; never expose
Project knowledge	Persistent instructions for the AI
Visual edit	Click-to-edit UI changes
Webhook	Message sent by a service when an event happens
Staging	Test environment

INTERVIEW PREP

12 interview questions with answers

Q1. What is Lovable?
An AI app builder that creates web apps from chat, with Supabase backend, GitHub sync and publishing.

Q2. How do you save user data?
Connect Supabase, create tables, and protect them with row-level security.

Q3. How do you take Lovable code further?
Sync to GitHub and continue with Claude Code, Codex or a developer.

Q4. How do you keep credits low?
Write a mini-PRD, batch related changes and be specific.

Q5. What is Lovable and who is it for?
An AI app builder that turns prompts into web apps with a live preview and Supabase backend. It suits PMs, founders and designers who want fast prototypes and MVPs.

Q6. What is Row Level Security and why is it important?
Rules in the database that control which rows each user can read or change. Without correct RLS, users could access each other's data.

Q7. How do you handle secrets in a Lovable app?
Keep private keys on the server side in edge functions and secrets; never in front-end code or prompts.

Q8. How do you write prompts that produce good results?
State role, goal, screens, data, rules and style; work in small slices; use examples and screenshots; say what must stay unchanged.

Q9. How do you take payments safely?
Use a hosted checkout, handle payment confirmation through a signed webhook on the server, store payment IDs, and never rely on the browser alone.

Q10. What do you test before publishing?
Happy path, edge cases, permissions with two users, mobile on a slow network, and the security scan results.

Q11. When should you hand over to engineers?
When there are real users, payments, sensitive data, scale or compliance needs, or logic beyond the generated code.

Q12. How do you manage credits and costs?
Be specific, work in slices, use visual edits and chat mode for questions, and check plan limits.

LEARN MORE

Further reading and tools

- Lovable Docs: https://docs.lovable.dev/
- Lovable prompting guide: https://docs.lovable.dev/prompting/prompting-one
- Supabase docs: https://supabase.com/docs
- Lovable community: https://discord.gg/lovable-dev
- Supabase: Row Level Security guide: https://supabase.com/docs/guides/database/postgres/row-level-security
- OWASP Top 10: https://owasp.org/www-project-top-ten/

© SkillNavora · skillnavora.in · Part of the AI Product Management and GenAI Launchpad programs. Practise with the free AI Tutor, AI PM Toolkit and Practice Lab on skillnavora.in.